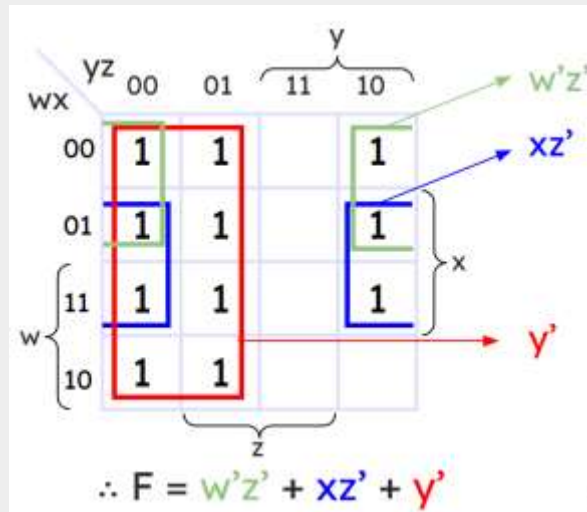
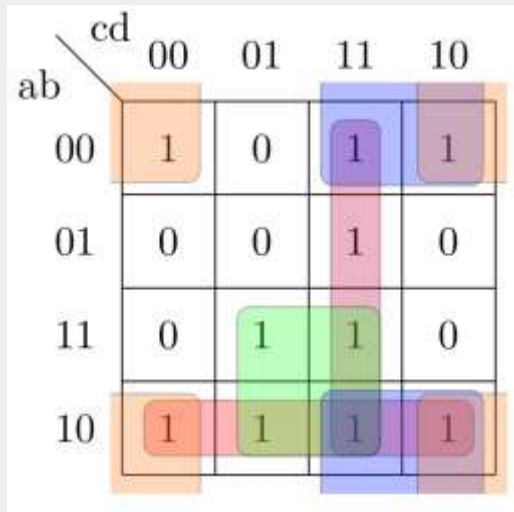


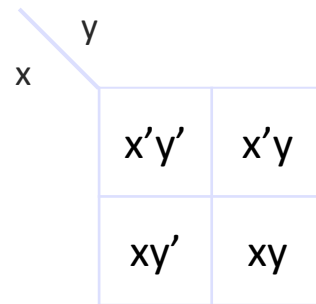
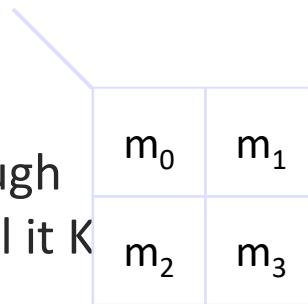
Lecture 5

Karnaugh Maps



The map method

- The map method provides a simple procedure for minimizing Boolean functions.
- The map is made up of squares where each square represent a minterm.
- We represent each minterm of an equation by '1's
- We groups the adjacent '1's in groups of 2, groups of 4 or groups of 8 and so on.
- In those grouping try to find the common literal.
- The map method is often known as Karnaugh Map or Veitch Diagram. In short we will call it K map

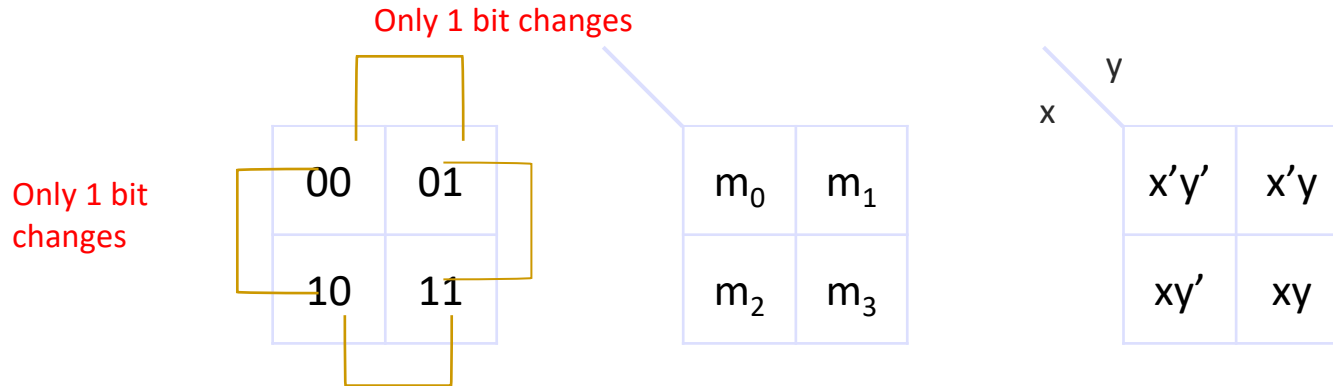


K Map Grouping Rules:

1. Group the adjacent '1's in groups of 2, 4, 8 and so on $[2^n]$.
2. Take maximum number of 1 if possible.
3. You can cover a 1 multiple times only if that covered 1 can help another uncovered 1 to get covered.
4. Our target is to group all the '1's.
5. In case of "don't care" we can consider 'X' to create groups alongside '1's. Other rules stay same as always.
6. However, you **cannot** create groups with 'X's only. 'X' can be only used to create groups if it is helping any uncovered '1's.

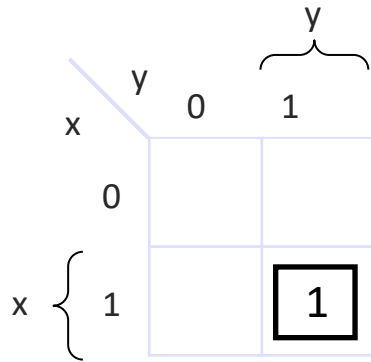
Two variable map

- There are 4 minterm of 2 variable; hence the map is four squares, one for each minterm.



Representing function in K-map

(a) $F1 = xy$;



(b) $F2 = xy' + xy + x'y$

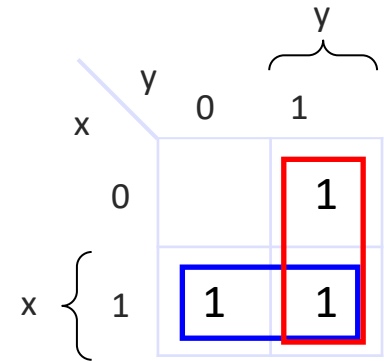
$$= x(y' + y) + x'y$$

$$= x + x'y$$

$$= (x + x')(x + y)$$

$$= (1)(x + y)$$

$$= \textcolor{blue}{x} + \textcolor{red}{y}$$



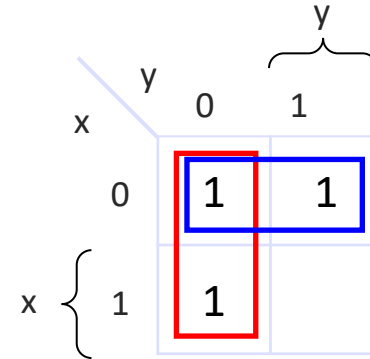
Try it yourself

$$F = xy' + x'y' + x'y,$$

Simplify the equation using k-map

Solution:

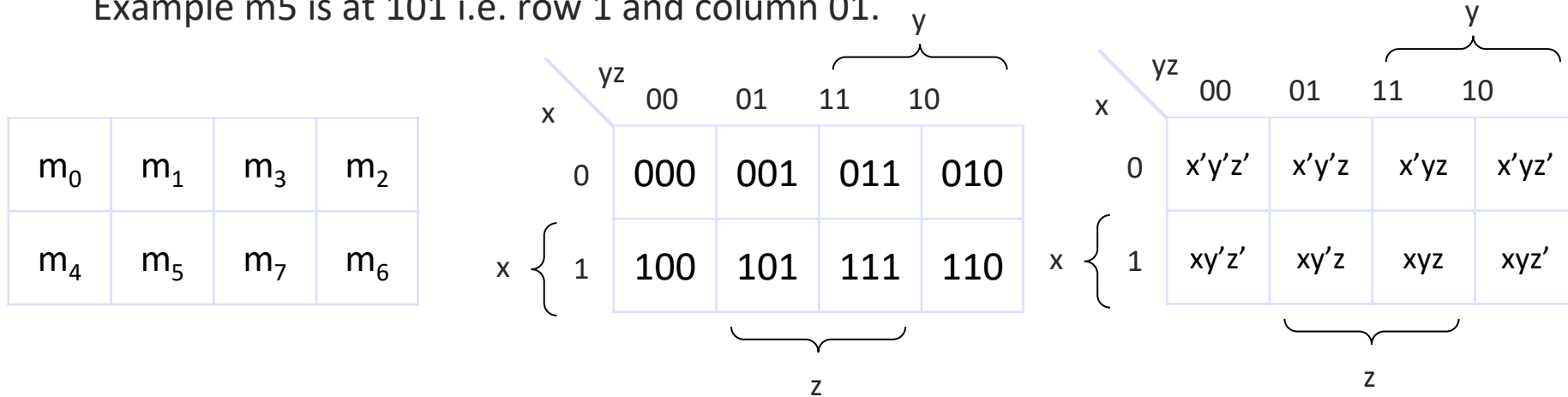
$$F = xy' + x'y' + x'y = x' + y'$$



Three variable map

- Sequence in 3-variable map is not in binary sequence but in reflected code. Here, at a time only 1 bit changes from 0 to 1 and 1 to 0.
- Numbering of minterms follow binary numbers.

Example m5 is at 101 i.e. row 1 and column 01.



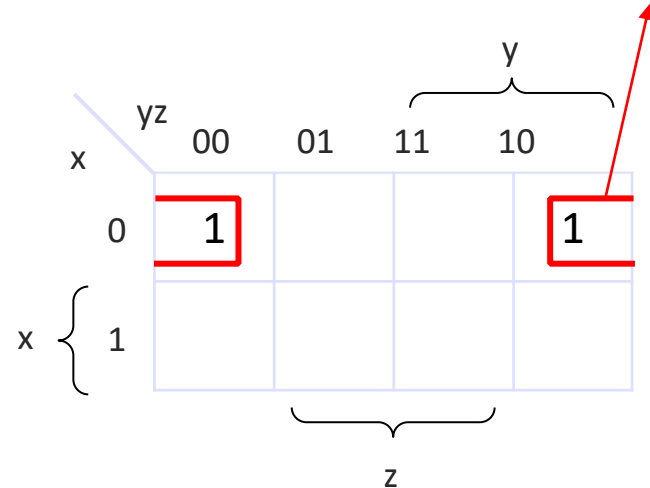
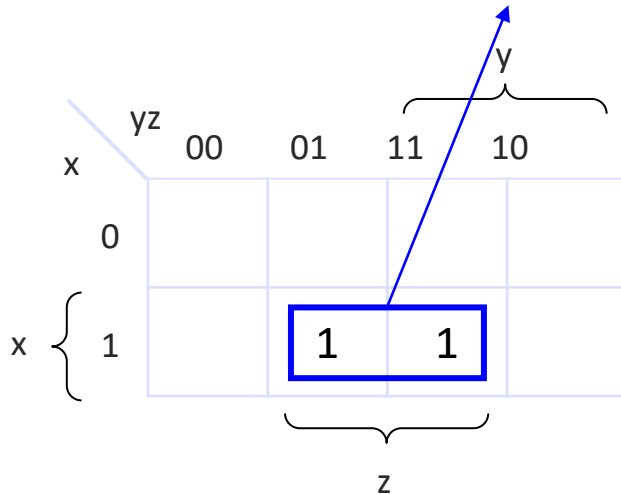
Note: Any 2 adjacent square differ by 1 variable (which is primed in one and unprimed in the other)

Three variable map

If any 2 adjacent square differ by 1 variable (which is primed in one and unprimed in the other) and they are grouped, then any 2 minterms in the adjacent square are **OR**ed, will remove the different term. As a result , we get a single **AND**ed term of only 2 common literals

$$F1 = \sum(5,7) = xy'z + xyz = xz(y' + y) = \mathbf{xz}$$

$$F2 = \sum(0,2) = x'y'z' + x'yz' = x'z'(y' + y) = \mathbf{x'z'}$$



Simplifying functions using k-map

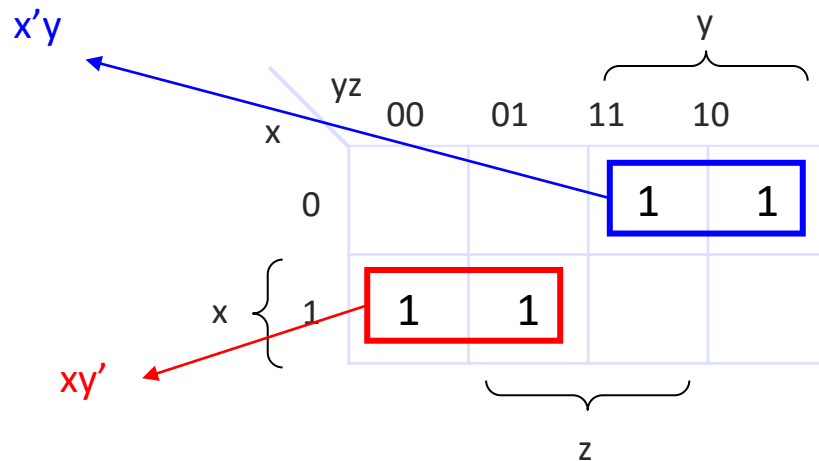
- $F = \sum(2,3,4,5)$

$$= x'yz + x'yz' + xy'z' + xy'z$$

(note: in other words, $F = (\sum(010,011,100,101))$ so place '1' in those positions and group the adjacent ones)

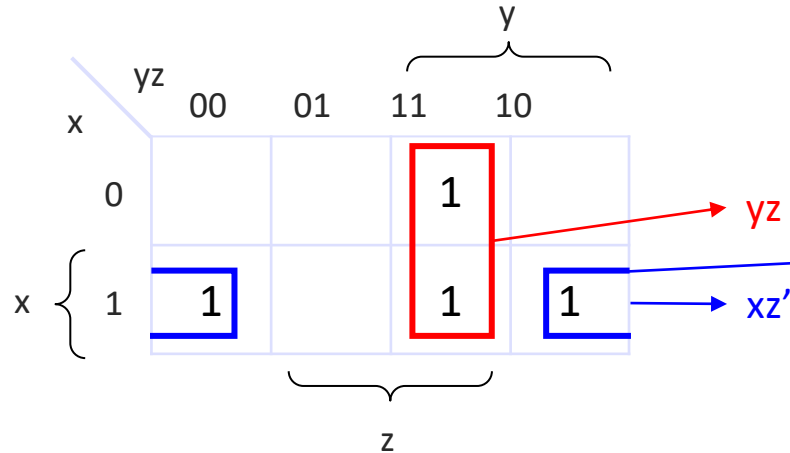
$$F = x'y + xy'$$

- Grouping of 1 can be done in groups of two '1', four '1', eight '1'.... 2^n '1'.



Example

$$F1 = \underset{011}{x'yz} + \underset{100}{xy'z'} + \underset{111}{xyz} + \underset{110}{xyz'} = \underset{110}{xz'} + \underset{011}{yz}$$



Notice even 2 opposite edges of the map are adjacent, thus they can be grouped

Example

If $F = \Sigma(0, 2, 4, 6)$, then simplify it

000

010

100

110

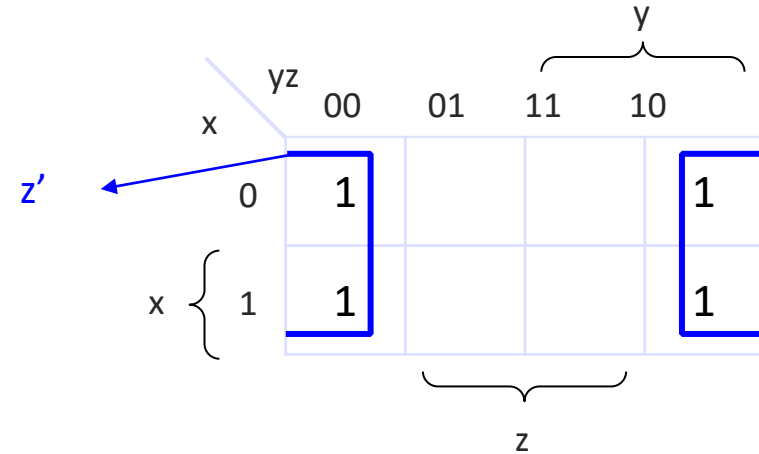
$$m_0 + m_2 + m_4 + m_6$$

$$= x'y'z' + x'yz' + xy'z' + xyz'$$

$$= x'z'(y' + y) + xz'(y' + y)$$

$$= x'z' + xz' = z'(x' + x)$$

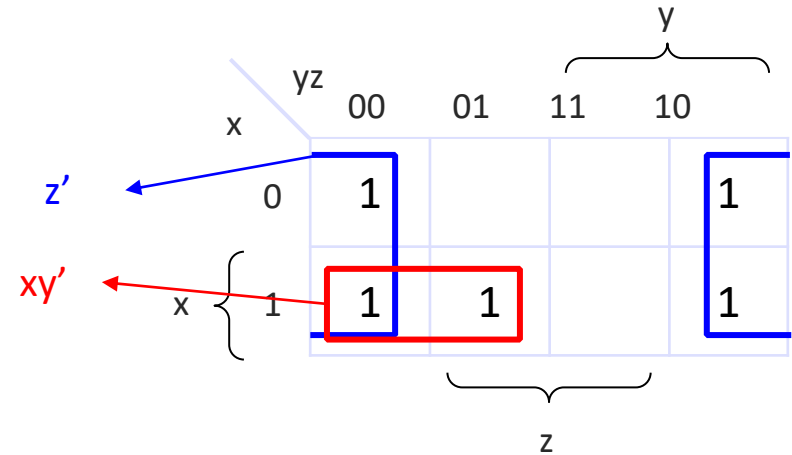
$$= z'$$



Try it yourself

$F = \sum(0, 2, 4, 5, 6)$, Simplify it.

Solution



$$F = xy' + z'$$

Try it yourself:

$$F = A'C + A'B + AB'C + BC$$

a) Express it as SOP (i.e. Σ)

b) Simplify it using Kmap

Solution

(a) $F = \Sigma(1, 2, 3, 5, 7)$

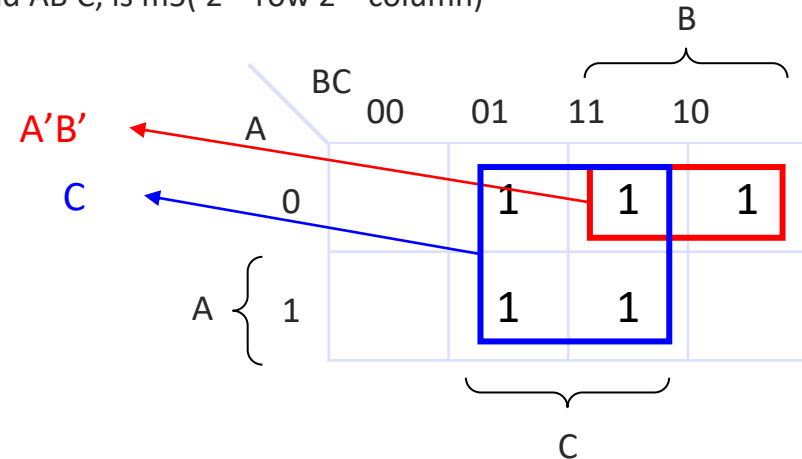
(b) $F = C + A'B$

To find $A'C$, coincide A' (first row) with C (middle 2 column)

To find $A'B$, coincide A' (first row) with B (last 2 column)

To find BC , coincide B (last 2 column) with C (middle 2 column)

To find $AB'C$, is m5(2nd row 2nd column)



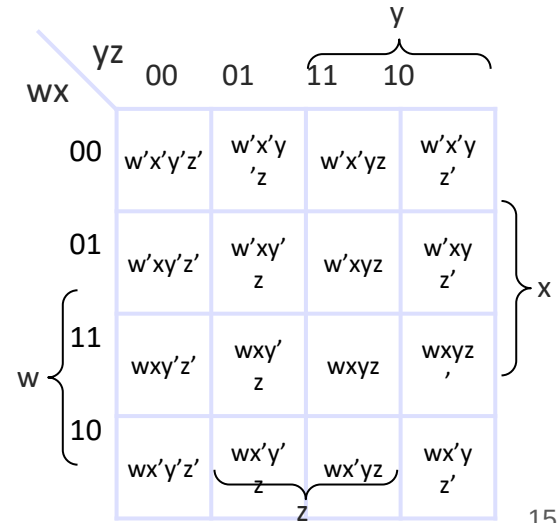
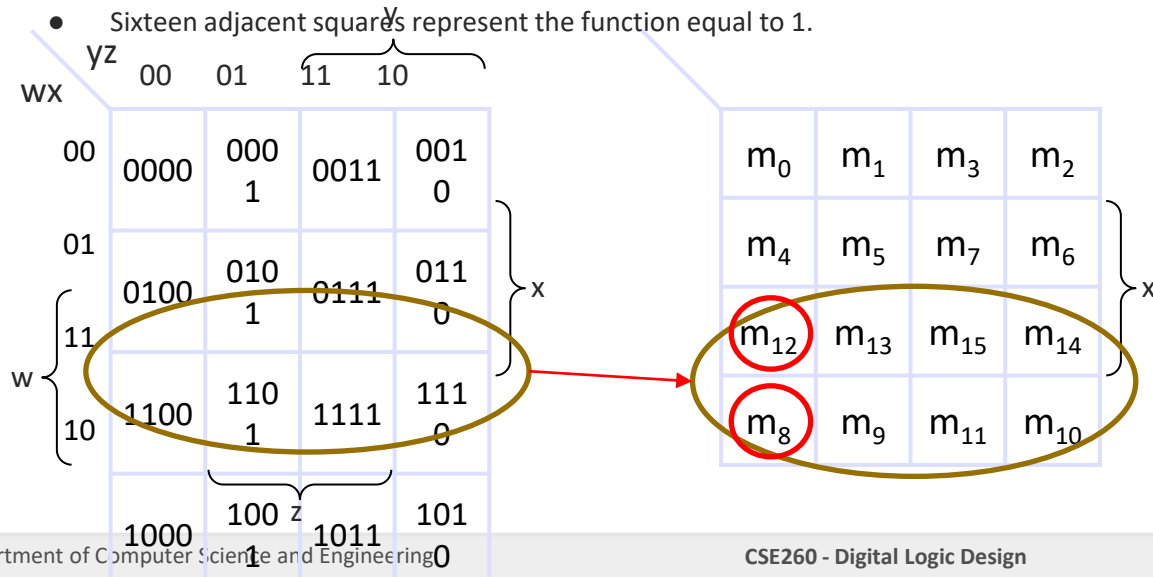
Summary of grouping in 3-variable map

- One square represents one minterm, giving a term of three literals
- Two adjacent squares represent a term of two literals
- Four adjacent squares represent a term of one literal
- Eight adjacent squares encompass the entire map and produce a function that is always equal to 1

		y			
		yz		11	10
x	0	$x'y'z'$	$x'y'z$	$x'yz$	$x'yz'$
	1	$xy'z'$	$xy'z$	xyz	xyz'
		z			

Four variable map

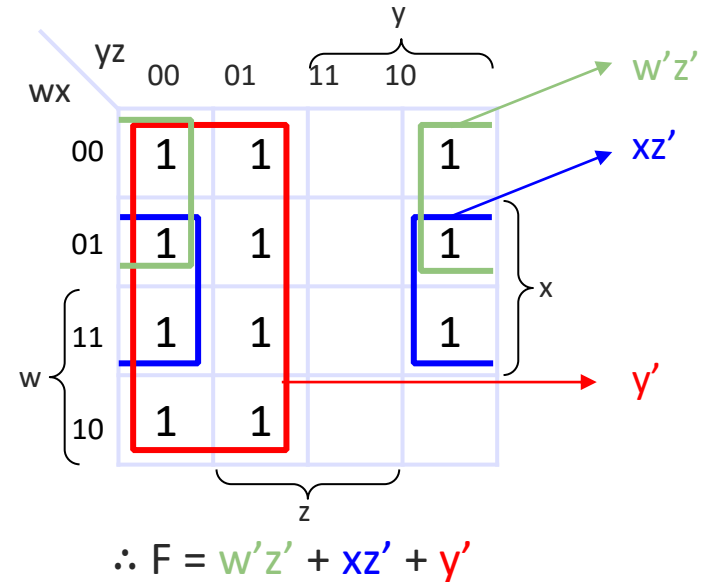
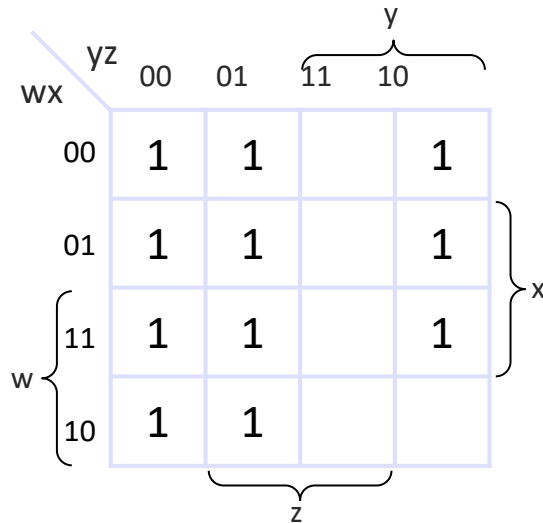
- Adjacent squares can be side by side or lie in 4 corner or top-bottom or left-right edge condition. Example, m_0 - m_2 are adjacent and similarly m_3 - m_{11} are adjacent
- One square represents one minterm, giving a term of four literals.
- Two adjacent squares represent a term of three literals.
- Four adjacent squares represent a term of two literals.
- Eight adjacent squares represent a term of one literal.
- Sixteen adjacent squares represent the function equal to 1.



Example

Simplify $F = \sum(0,1,2,4,5,6,8,9,12,13,14)$

Note: it is allowed to use the same '1' more than once.



Try it yourself

$$F = A'B'C' + B'CD' + A'BCD' + AB'C'$$

		C			
		CD		11	10
A	AB	00	01		
	00	1	1		1
	01				1
	11				
	10	1	1		1

The Karnaugh map is a 4x4 grid with rows labeled AB (00, 01, 11, 10) and columns labeled CD (00, 01, 11, 10). The map is partitioned into four groups of four cells each, each containing 1s:

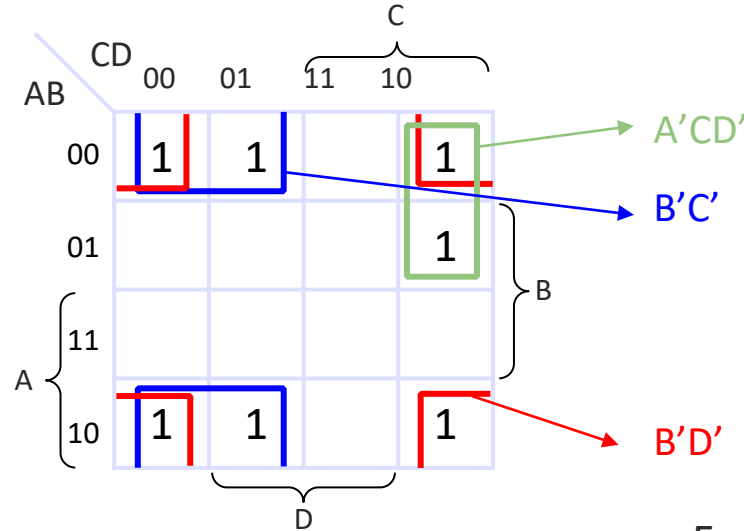
- Group 1 (Top-left): AB=00, CD=00 and 01.
- Group 2 (Top-right): AB=00 and 01, CD=10.
- Group 3 (Bottom-left): AB=10, CD=00 and 01.
- Group 4 (Bottom-right): AB=10 and 11, CD=10.

 The groups are labeled with curly braces: 'C' for the top two groups, 'B' for the right two groups, 'A' for the left two groups, and 'D' for the bottom two groups.

Try it yourself

$$F = A'B'C' + B'CD' + A'BCD' + AB'C'$$

Solution



$$\therefore F = A'CD' + B'C' + B'D'$$

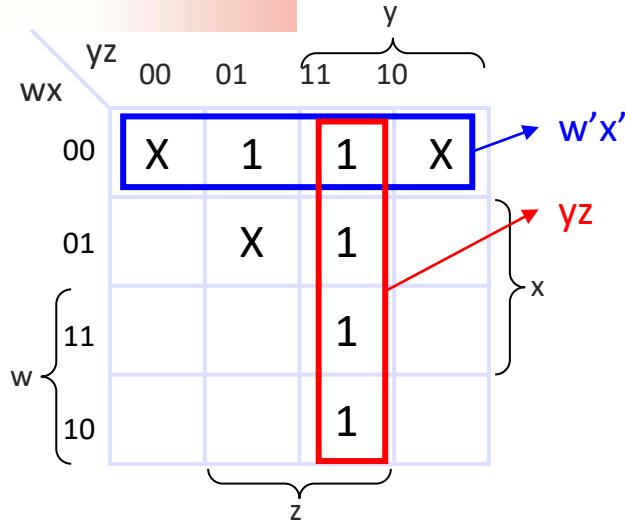
Don't Care condition

- Don't cares in a Karnaugh map, or truth table, may be either 1s or 0s, as long as we don't care what the output is for an input condition we never expect to see.
- We plot these cells with a special sign, 'X', among the normal 1s and 0s.
- When forming groups of cells, treat the don't care cell as either a 1 or a 0, or ignore the don't cares. This is helpful if it allows us to form a larger group than would otherwise be possible without the don't cares.
- There is no requirement to group all or any of the don't cares. (i.e. do not make any group only consisting the don't cares). Only use them in a group if it simplifies the logic.

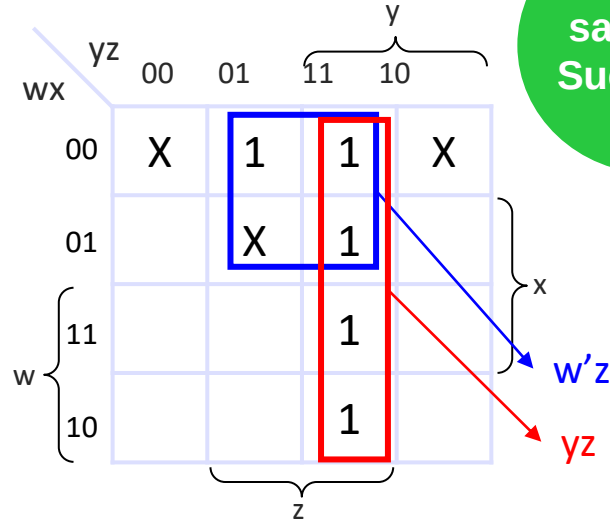
Example

Simplify $F(w,x,y,z) = \sum(1,3,7,11,15)$ and don't care $d(w,x,y,z) = \sum(0,2,5)$

Solution:



$$\therefore F = w'x' + yz$$



$$\therefore F = w'z + yz$$

Even though 2 solutions are not same, but either 1 is acceptable. Such scenario only applicable for Don't care scenarios!



APPLICATIONS OF K-MAP

Example

1. Design and draw the circuit diagram of BCD to Excess-3 code converter

- Treat each W, X, Y and Z as a function and write them as SOP
- Based on the SOP, draw the circuit diagram of BCD to Excess-3 code converter

Any value higher than 9 will result into don't care output

BCD Input				Excess-3 Output			
B3	B2	B1	B0	W	X	Y	Z
0	0	0	0	0	0	1	1
0	0	0	1	0	1	0	0
0	0	1	0	0	1	0	1
0	0	1	1	0	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	0	0	0
0	1	1	0	1	0	0	1
0	1	1	1	1	0	1	0
1	0	0	0	1	0	1	1
1	0	0	1	1	1	0	0
1	0	1	0	x	x	x	x
1	0	1	1	x	x	x	x
1	1	0	0	x	x	x	x
1	1	0	1	x	x	x	x
1	1	1	0	x	x	x	x
1	1	1	1	x	x	x	x

Example 1 [Cont.]

For W

CD \ AB	00	01	11	10
00				
01		1	1	1
11	X	X	X	X
10	1	1	X	X

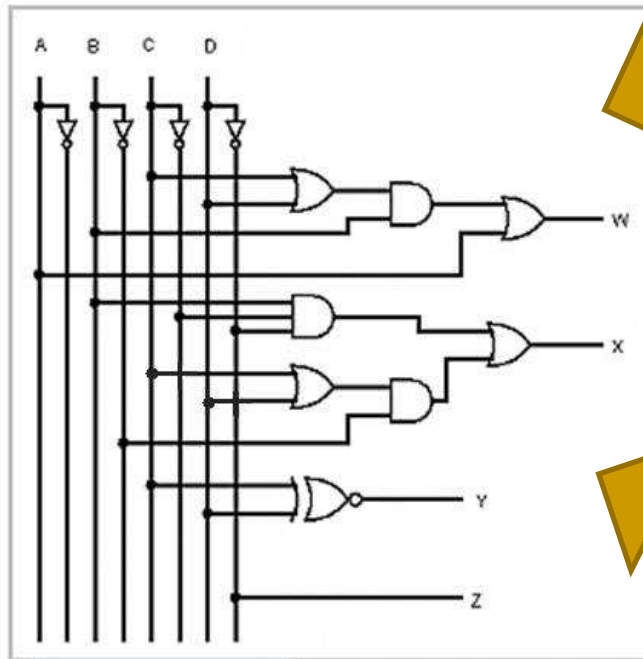
$$W = A + BC + BD$$

$$= A + B(C + D)$$

For Z

CD \ AB	00	01	11	10
00	1			1
01	1			1
11	X	X	X	X
10	1			X

$$Z = \overline{D}$$



For X

CD \ AB	00	01	11	10
00		1	1	1
01	1			
11	X	X	X	X
10		1	X	X

$$X = B\overline{C}\overline{D} + \overline{B}(C + D)$$

For Y

CD \ AB	00	01	11	10
00	1		1	
01	1		1	
11	X		X	X
10	1		X	X

$$Y = \overline{C}\overline{D} + CD$$

Example

2: 7 segment display board

Grey represents that segment is active = 1;

White represents that segment is inactive = 0

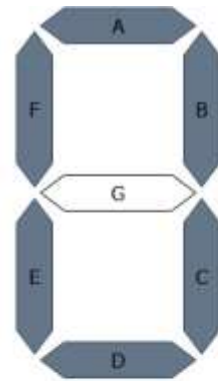


Figure: 0

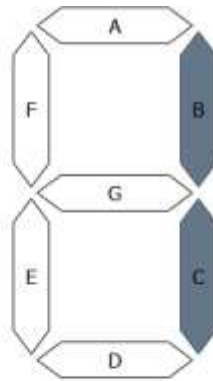


Figure: 1

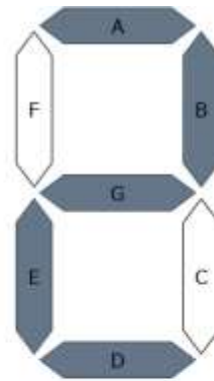


Figure: 2

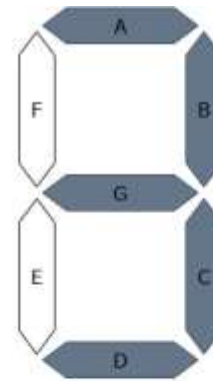


Figure: 3

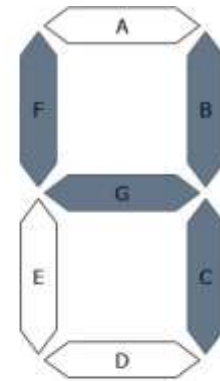


Figure: 4

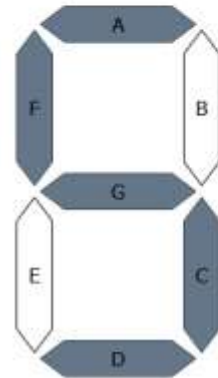


Figure: 5

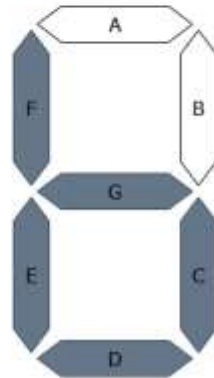


Figure: 6

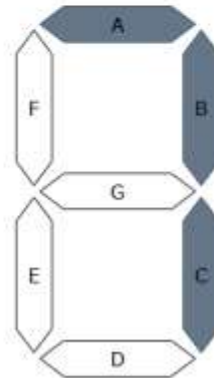


Figure: 7

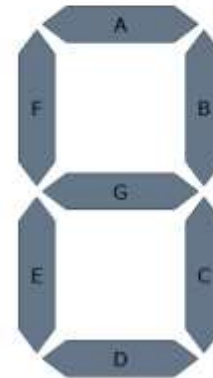


Figure: 8

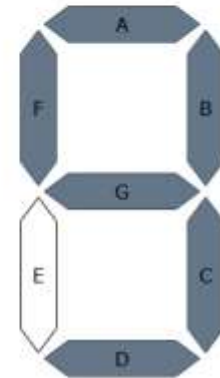


Figure: 9

Example

2: 7 segment display board

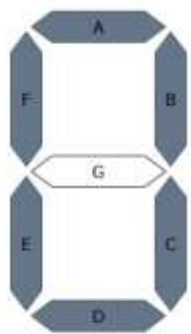


Figure: 0

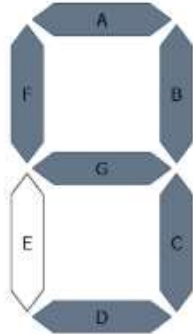
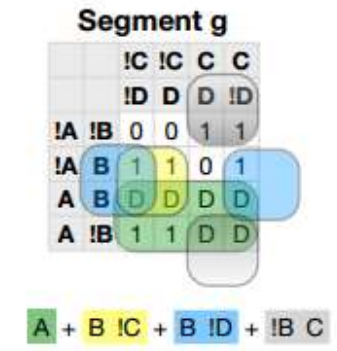
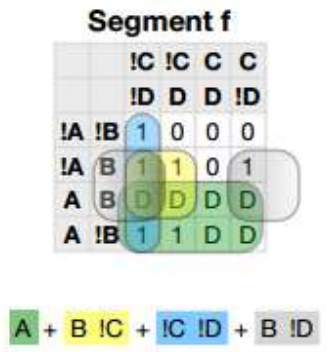
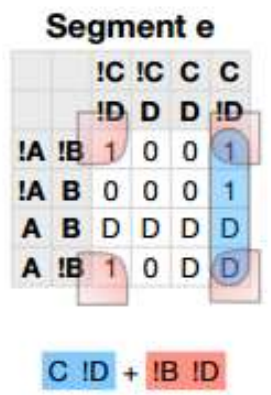
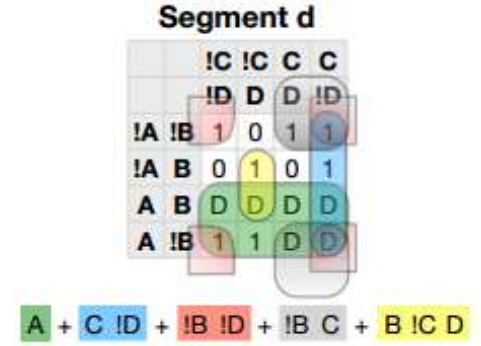
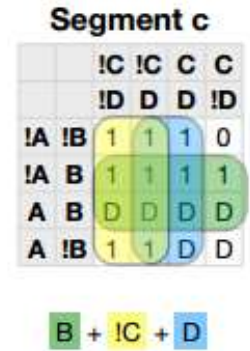
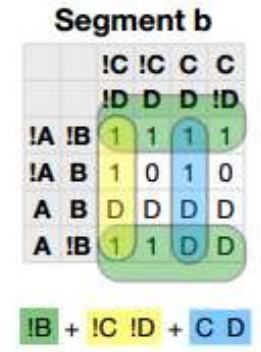
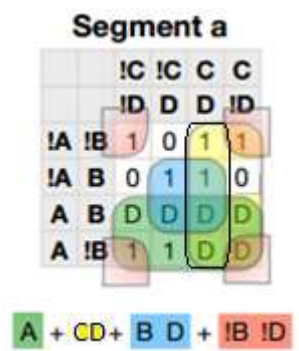


Figure: 9

Display Digit	b8	b4	b2	b1	A	B	C	D	E	F	G
0	0	0	0	0	1	1	1	1	1	1	0
1	0	0	0	1	0	1	1	0	0	0	0
2	0	0	1	0	1	1	0	1	1	0	1
3	0	0	1	1	1	1	1	1	0	0	1
4	0	1	0	0	0	1	1	0	0	1	1
5	0	1	0	1	1	0	1	1	0	1	1
6	0	1	1	0	0	0	1	1	1	1	1
7	0	1	1	1	1	1	1	0	0	0	0
8	1	0	0	0	1	1	1	1	1	1	1
9	1	0	0	1	1	1	1	1	0	1	1
off	1	0	1	0	x	x	x	x	x	x	x
off	1	0	1	1	x	x	x	x	x	x	x
off	1	1	0	0	x	x	x	x	x	x	x
off	1	1	0	1	x	x	x	x	x	x	x
off	1	1	1	0	x	x	x	x	x	x	x
off	1	1	1	1	x	x	x	x	x	x	x

Example 2 [Cont.]





Example 2 [Cont.]

Draw the circuit
yourself.....

Try it yourself

- A car garage has a front door and one window, each of which has a sensor to detect whether it is open. A third sensor detects whether it is dark outside. A security system for the garage follows this rule:

the alarm rings if the **alarm switch is turned on** and **the side window is not closed** and **either the front door is not closed or it is dark**

Solution

Assuming

AlarmRing = 1 if Alarm rings, 0 otherwise.

Switch On = 1 if alarm switch is on, 0 otherwise.

Door = 1 if door is not closed, 0 otherwise.

Window = 1 if window is not closed.

Dark = 1 if it is dark outside, 0 otherwise.

SwitchOn	Door	Window	Dark	AlarmRing
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	0
1	1	1	0	1
1	1	1	1	1

Solution

SwitchOn	Door	Window	Dark	AlarmRing
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	0
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	0
1	1	1	0	1
1	1	1	1	1

WDr

SDr

		1	1
		1	

$$\text{AlarmRing} = \text{SWDr} + \text{SDrW}$$

Next Step

- You do have to draw the circuit for the above function
- You may be asked to draw the circuit using basic gates or universal gates
- By now, you should be able to do it yourself. So please try.